School of Computer Science Engineering & Information Systems

Department of Software & Systems Engineering

ISWE406P – Agile Development Process and Devops Lab

ASSESSMENT -1

NAME : SARAVANAN.R

REGISTER NO : 23MIS0460

FACULTY NAME : DR . RAMALINGAM M

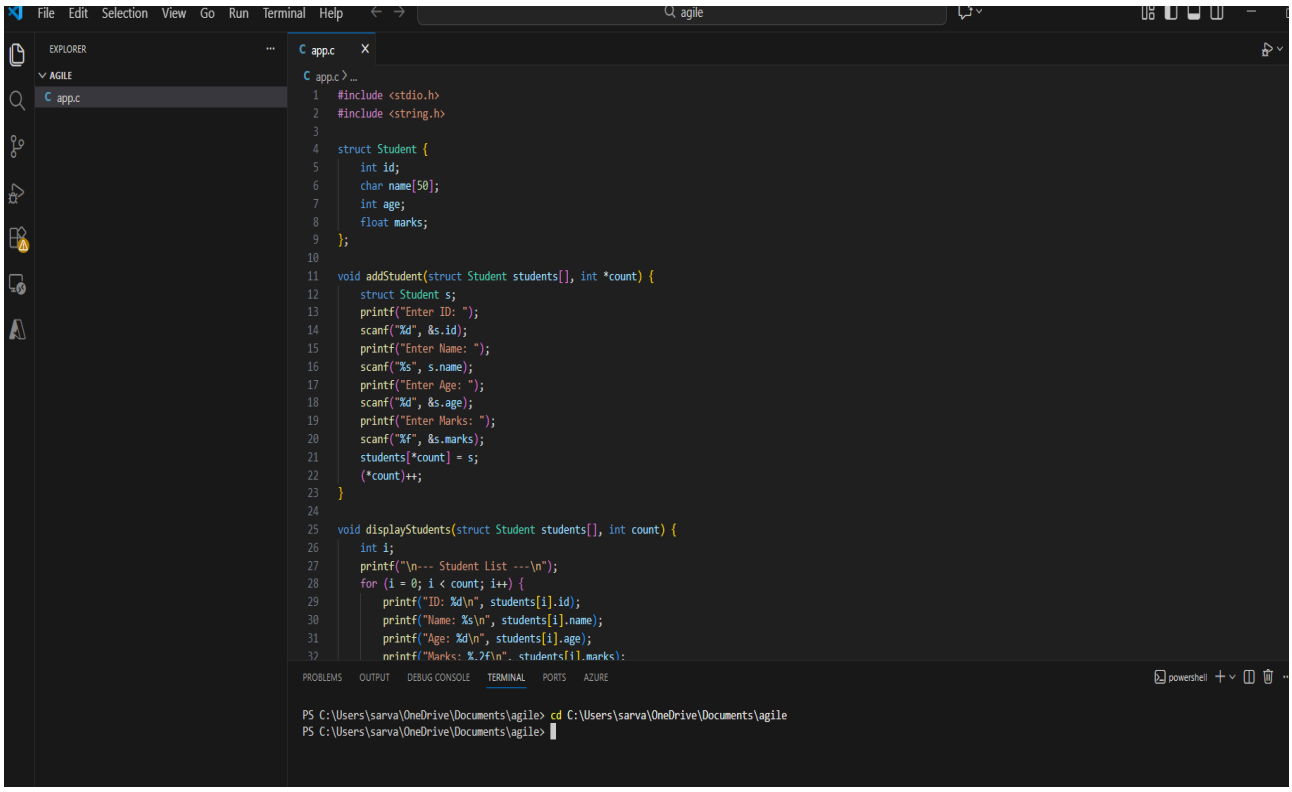
Experiment 1: Develop a simple application (console-based or GUI-based) using any programming language of your choice. Initialize a local Git repository for the project and manage the source code using Git version control. Commit the application source code with proper commit messages and maintain version history.

Create a remote repository on GitHub and push the local repository to GitHub. Verify successful hosting by accessing the repository online.

Perform branch-related operations by creating at least one feature branch from the main branch, implement a minor enhancement or modification in the feature branch, commit the changes, and merge the feature branch back into the main branch. Resolve any merge conflicts if they arise and push the updated main branch to GitHub.

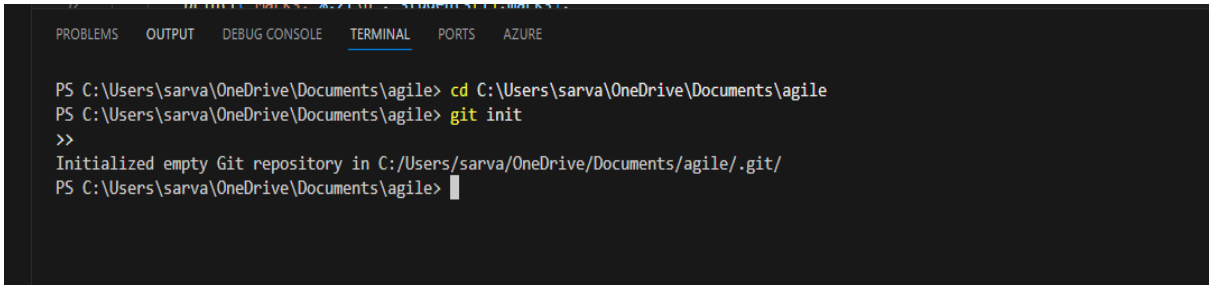
Step 1: Create Project Folder

1. Create a new folder named GitLabExperiment.
2. Open the folder in VS Code.
3. Create a file named app.c

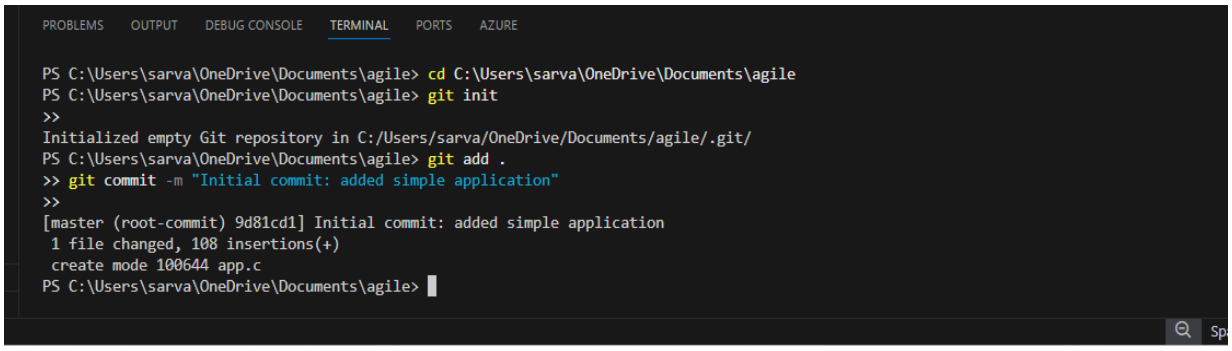


Step 2: Initialize Git Repository

Open terminal inside the project folder and run:

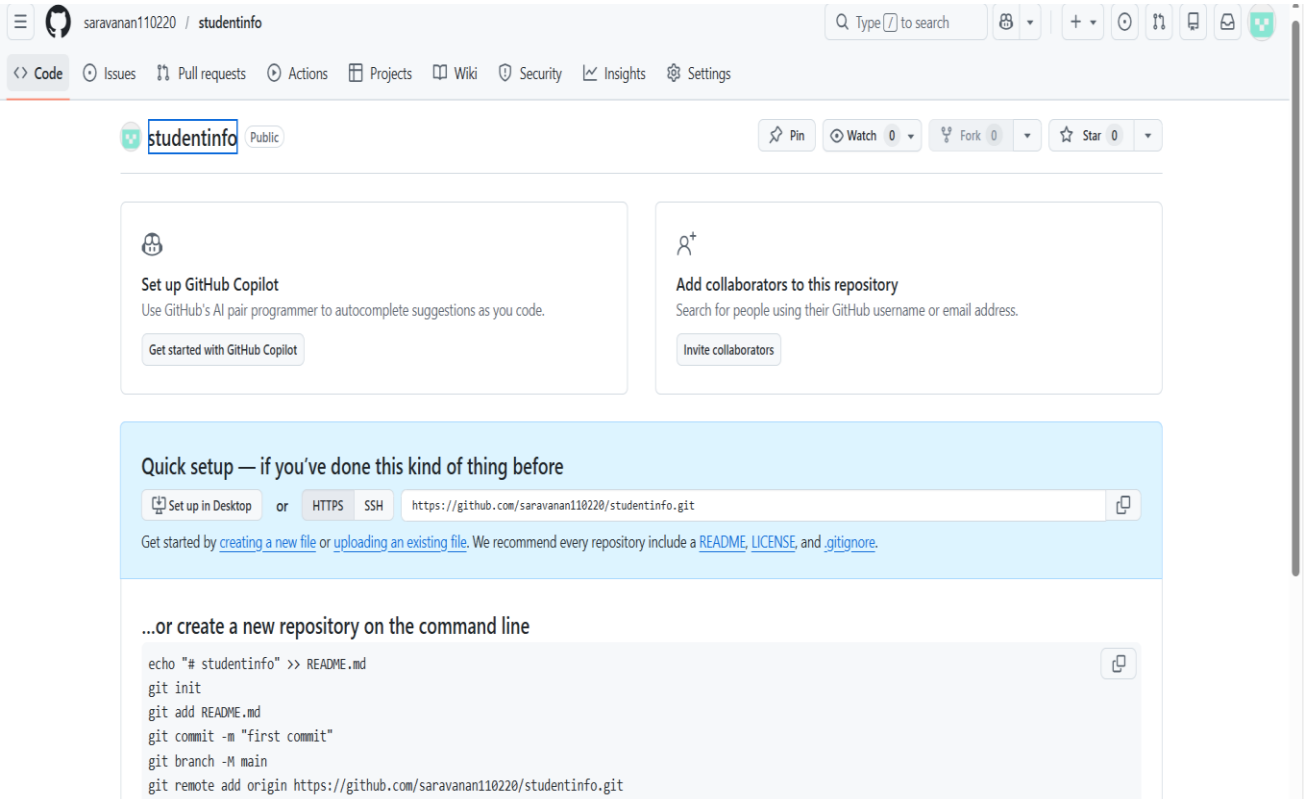


Step 3: Add Files and First Commit



Step 4: Create GitHub Repository

1. Open GitHub → New Repository
2. Repository name: StudentInfo
3. Select Public
4. Click Create Repository

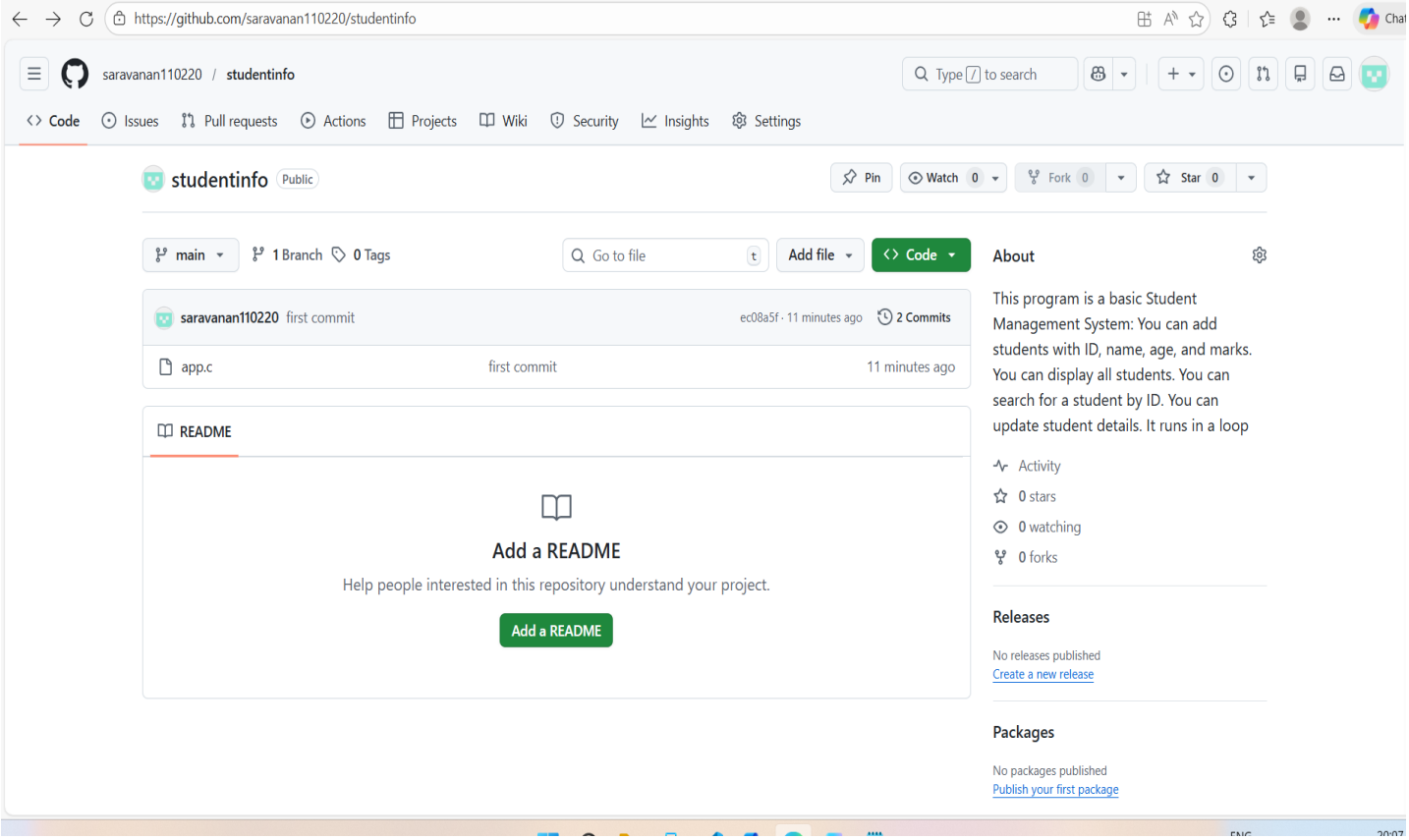


Step 5: Push Local Repository to GitHub

git branch -M main

git remote add origin https://github.com/USERNAME/GitLabExperiment.git

git push -u origin main



Step 6: Create a Feature Branch

git checkout -b feature-enhancement

```
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/saravanan110220/studentinfo.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\sarva\OneDrive\Documents\agile> git checkout -b feature-enhancement
>>
Switched to a new branch 'feature-enhancement'
PS C:\Users\sarva\OneDrive\Documents\agile>
```

Step 7: Modify Application in Feature Branch

```c
void averageMarks(struct Student students[], int count) {
    if (count == 0) {
        printf("No students available.\n");
        return;
    }
    float sum = 0;
    for (int i = 0; i < count; i++) {
        sum += students[i].marks;
    }
    printf("Average Marks: %.2f\n", sum / count);
}

void topperStudent(struct Student students[], int count) {
    if (count == 0) {
        printf("No students available.\n");
        return;
    }
    int topIndex = 0;
    for (int i = 1; i < count; i++) {
        if (students[i].marks > students[topIndex].marks) {
            topIndex = i;
        }
    }
```

Step 8: Commit Feature Branch Changes

git add .

git commit -m "Added average marks feature"

```
Switched to a new branch 'feature-enhancement'
PS C:\Users\sarva\OneDrive\Documents\agile> git add .
>> git commit -m "Added average marks feature"
>>
>>
[feature-enhancement 868e62b] Added average marks feature
 1 file changed, 64 insertions(+), 2 deletions(-)
PS C:\Users\sarva\OneDrive\Documents\agile>
```

Step 9: Merge Feature Branch with Main

```
PS C:\Users\sarva\OneDrive\Documents\agile> git checkout main
>> git merge feature-enhancement
>>
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
Updating ec08a5f..868e62b
Fast-forward
 app.c | 66 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++--
 1 file changed, 64 insertions(+), 2 deletions(-)
PS C:\Users\sarva\OneDrive\Documents\agile>
```

Step 10: Push Updated Main Branch

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   AZURE

PS C:\Users\sarva\OneDrive\Documents\agile> git push origin main
>>
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 775 bytes | 775.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/saravanan110220/studentinfo.git
   ec08a5f..868e62b  main -> main
PS C:\Users\sarva\OneDrive\Documents\agile>
Ln 171, Col 1   Spaces: 4   UTF-8
```

saravanan110220 / studentinfo

Type / to search

Code | Issues | Pull requests | Actions | Projects | Wiki | Security | Insights | Settings

Commits

main

All users | All time

Commits on Jan 9, 2026

Added average marks feature
saravanan110220 committed 2 minutes ago
868e62b

first commit
saravanan110220 committed 27 minutes ago
ec08a5f

https://github.com/saravanan110220/studentinfo/blob/main/app.c

Files

main

Go to file

app.c

studentinfo / app.c

Top

Code | Blame | 170 lines (161 loc) · 4.49 KB

Raw

```c
struct Student {
    if (!found) {
        printf("Student not found.\n");
    }
}

void averageMarks(struct Student students[], int count) {
    if (count == 0) {
        printf("No students available.\n");
        return;
    }
    float sum = 0;
    for (int i = 0; i < count; i++) {
        sum += students[i].marks;
    }
    printf("Average Marks: %.2f\n", sum / count);
}

void topperStudent(struct Student students[], int count) {
    if (count == 0) {
        printf("No students available.\n");
        return;
    }
    int topIndex = 0;
    for (int i = 1; i < count; i++) {
        if (students[i].marks > students[topIndex].marks) {
            topIndex = i;
        }
    }
    printf("Topper:\n");
    printf("ID: %d\n", students[topIndex].id);
    printf("Name: %s\n", students[topIndex].name);
```

Experiment 2: Form a team of five members to collaboratively develop a simple application using **Git and GitHub**. One member shall act as the **repository owner**, while the remaining members shall act as **collaborators.**

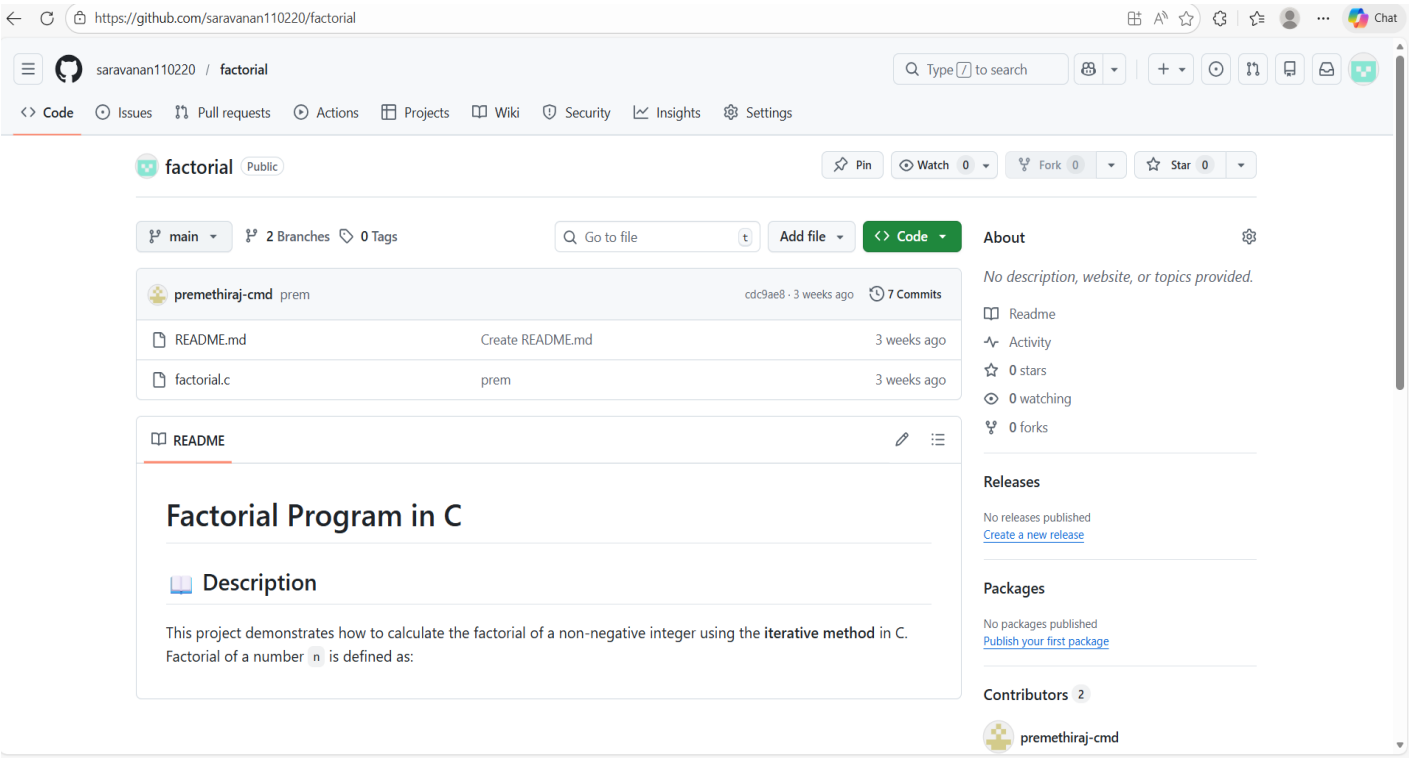
Create a shared GitHub repository and clone it to all team members' local systems. Each team member must create a separate feature branch from the main branch and modify the **same source file** by adding their own functionality or contribution.

Each collaborator shall commit and push their changes to their respective branches and raise a pull request to the main branch. The repository owner shall review the pull requests, resolve any merge conflicts that arise due to simultaneous modification of the same file, and merge the changes into the main branch.

Finally, push the updated main branch to GitHub and verify that the final application includes contributions from all five team members. Document the Git commands used and explain the collaborative workflow followed in the experiment.

Step 1: Create GitHub Repository (Repository Owner)

1. Login to GitHub
2. Click **New Repository**
3. Repository name: factorial
4. Set to **Public**
5. Click **Create Repository**



Step 2: Cloning the Repository

The shared repository was cloned to the local system using Visual Studio Code terminal.

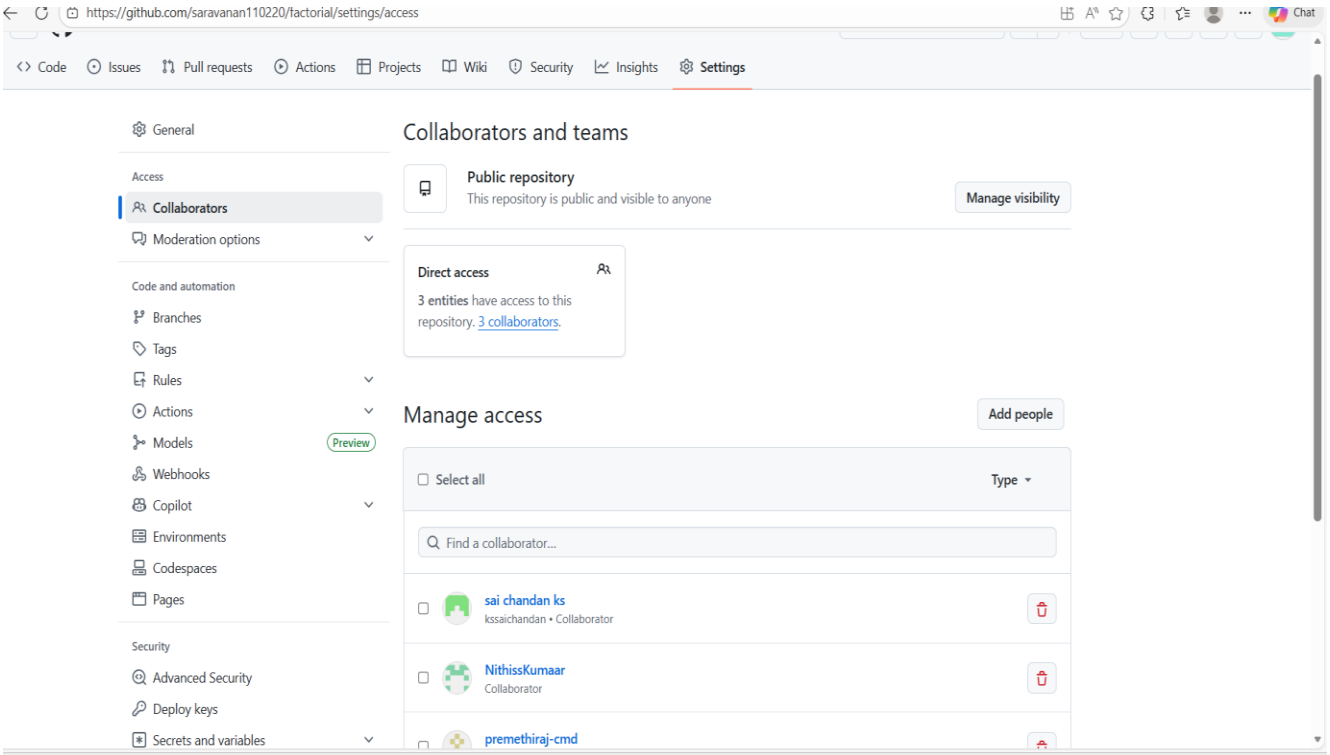
git clone https://github.com/saravanan110220/factorial.git

cd factorial

```
PS C:\Users\sarva\OneDrive\Documents\agile> git clone https://github.com/saravanan110220/factorial.git
>> cd factorial
>>
Cloning into 'factorial'...
remote: Enumerating objects: 21, done.
remote: Counting objects: 100% (21/21), done.
remote: Compressing objects: 100% (13/13), done.
remote: Total 21 (delta 5), reused 17 (delta 4), pack-reused 0 (from 0)
Receiving objects: 100% (21/21), done.
Resolving deltas: 100% (5/5), done.
PS C:\Users\sarva\OneDrive\Documents\agile\factorial>
```

Step 3: Adding Collaborators

Collaborators were added to the repository through **Settings → Collaborators.**
The invited members accepted the collaboration request successfully.



Step 4: Creating Feature Branches

Each team member created an individual feature branch from the main branch to work independently.

```
PS C:\Users\sarva\OneDrive\Documents\agile\factorial> git checkout -b prem
>>
Switched to a new branch 'prem'
PS C:\Users\sarva\OneDrive\Documents\agile\factorial>
```

Step 5: Modifying the Same Source File

All collaborators modified the same source file **factorial.c** by adding their own logic or contribution related to the factorial program**.**

Step 6: Committing Changes

After modification, the changes were staged and committed with proper commit messages.

git add factorial.c

git commit -m "Added factorial logic by collaborator"

```
>> git commit -m "Added factorial logic by collaborator"
>>
On branch prem
nothing to commit, working tree clean
PS C:\Users\sarva\OneDrive\Documents\agile\factorial>
```

Step 7: Pushing Feature Branch to GitHub

Each collaborator pushed their respective feature branch to the remote repository.

```
PS C:\Users\sarva\OneDrive\Documents\agile\factorial> git push origin prem
>>
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'prem' on GitHub by visiting:
remote:      https://github.com/saravanan110220/factorial/pull/new/prem
remote:
To https://github.com/saravanan110220/factorial.git
 * [new branch]      prem -> prem
PS C:\Users\sarva\OneDrive\Documents\agile\factorial>
Ln 171,
```

Step 8: Raising Pull Requests

Pull Requests were created from the feature branches to the main branch using GitHub’s Pull Request feature.

Step 9: Reviewing and Merging Pull Requests

The repository owner reviewed the pull requests.
Any merge conflicts due to simultaneous file modification were resolved, and all pull requests were successfully merged into the main branch.

Step 10: Final Verification

The updated main branch was pulled and verified to ensure that all collaborators' contributions were present in the final application.

git checkout main

git pull origin main

```
PS C:\Users\sarva\OneDrive\Documents\agile\factorial> git checkout main
>> git pull origin main
>>
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
From https://github.com/saravanan110220/factorial
 * branch            main       -> FETCH_HEAD
Already up to date.
PS C:\Users\sarva\OneDrive\Documents\agile\factorial>
```